Reading Assignment - Lab 03

Object-Oriented Programming · Trinh Huu Khanh · 202517015 · ICT

1. What are the advantages of Polymorphism?

Polymorphism ("many forms") means that one interface - a method call on a supertype reference - can trigger different behaviour depending on the actual object. In Java it appears as **compile-time polymorphism** (method overloading, resolved by the compiler from the argument types) and **run-time polymorphism** (method overriding, resolved by the JVM through dynamic binding). Its main advantages are:

- **Code that works with the general type.** A method written against `Media` or `Playable` works for every current and future subtype. In AIMS, `Cart` keeps one `ArrayList<Media>` and one `addMedia()` instead of separate methods for DVDs, Books and CDs.
- **Extensibility (Open/Closed principle).** Adding a new media type only requires a new subclass that overrides `toString()` or `play()`; Cart, Store and the menus do not change.
- **No type-checking chains.** Long `if (x instanceof DVD) ... else if (x instanceof CD)` blocks are replaced by a single call such as `m.toString()`, which is shorter and less error-prone.
- **Reusability and less duplication.** Shared logic (total cost, sorting with a comparator, searching by title) is written once for the supertype and reused by every subclass.
- **Loose coupling and easier maintenance/testing.** Callers depend on an abstraction, not on concrete classes, so implementations can be swapped or mocked without touching the callers.
- **Readable, natural code.** Overloading lets related operations share one meaningful name, e.g. several `DigitalVideoDisc` constructors or `println(int)` / `println(String)`.

```
List<Media> mediae = new ArrayList<Media>();
mediae.add(cd); mediae.add(dvd); mediae.add(book);
for (Media m : mediae) {
    System.out.println(m.toString()); // CD, DVD or Book version chosen at run time
}
```

2. How is Inheritance useful to achieve Polymorphism in Java?

Run-time polymorphism in Java is built directly on inheritance (class inheritance with `extends` or interface implementation with `implements`). Inheritance supplies the three ingredients polymorphism needs:

- **An IS-A relationship and upcasting.** Because `DigitalVideoDisc extends Disc extends Media`, a DVD *is a* Media, so it can be stored in a `Media` variable or a `List<Media>`. Without a common supertype there is no single reference type that can point to different objects.
- **A common contract.** The superclass (or interface) declares the methods every subtype is guaranteed to have - `toString()` from `Object`/`Media`, `play()` from `Playable`. The compiler only allows the call because the supertype declares it.
- **Overriding.** Subclasses inherit the method and redefine it with the same signature. At run time the JVM looks at the object's real class (dynamic dispatch through the method table) and runs

the most specific version.

- **Abstract classes and interfaces.** An abstract method (or an interface method such as `Playable.play()`) forces every concrete subclass to provide its own behaviour, so the polymorphic call is always valid.

In short: inheritance creates the type hierarchy and the shared method signatures; overriding plus dynamic binding turns that hierarchy into polymorphic behaviour. Interfaces even allow unrelated classes to share polymorphic behaviour - `Track` is not a `Media`, yet it is `Playable` just like `CompactDisc` and `DigitalVideoDisc`.

3. What are the differences between Polymorphism and Inheritance in Java?

They are closely related but answer different questions: inheritance is about **how classes are built** (reusing and extending a parent), polymorphism is about **how objects behave** when used through a general type.

Criterion	Inheritance	Polymorphism
Definition	A class acquires the fields and methods of another class (child extends parent).	One interface/method call takes many forms; behaviour depends on the actual object or the arguments.
Purpose	Code reuse and modelling an IS-A hierarchy.	Flexibility: write general code that works with many types.
Applies to	Classes and interfaces (the structure of types).	Methods / behaviour (how calls are resolved).
Java keywords / mechanism	extends, implements, super	Overriding (+ dynamic binding), overloading
Binding time	Defined at compile time when the class is declared.	Overloading: compile time. Overriding: run time.
Forms	Single, multilevel, hierarchical (multiple only via interfaces).	Compile-time (static) and run-time (dynamic).
Dependency	Can exist without polymorphism (just reusing fields like id, title, cost).	Run-time polymorphism requires inheritance or an interface; overloading does not.
AIMS example	Book and Disc extend Media and reuse id, title, category, cost.	m.toString() / p.play() run the CD, DVD, Book or Track version.

Conclusion. Inheritance is the means and polymorphism is one of its most important results: inheritance builds the Media hierarchy and lets subclasses reuse common code, while polymorphism lets Cart, Store and the console menu treat every item simply as a `Media` (or `Playable`) and still get the correct, type-specific behaviour.